

Lecture 13: Node.js

실습 (Practice Worksheet)

이 자료는 강의노트 **Lecture 13 (Node.js)**의 모든 내용을 직접 구현해 보는 실습 워크시트입니다.

각 단원은 ① 개념 복습(Practice) → ② 과제(Task) → ③ 예상 결과 순서이며, 과제는 앞 과제 위에 누적됩니다.

정답 코드는 실지 않습니다. 강의 페이지([00p])를 참고해 스스로 채워 넣으세요.

In the Last Lecture (지난 강의 복습)

- **React** — JSX, 컴포넌트, props, Hooks(useState/useEffect/useRef), Virtual DOM
- 이번 강의: 그 React **프론트엔드에 데이터를 보내줄 백엔드**를 Node.js로 만든다.

Introduction to Node.js

개념 · 구현 없음

개념 정리 [1~10p]

- 백엔드 = 서버 측. 데이터·로직·인증·DB 통신 담당 [4p]
- Node.js = V8 엔진 + 서버 기능 → 브라우저 밖에서 JS 실행 [5p]
- Node로 할 수 있는 것: 서버 생성 / DB 연결 / 파일 입출력 / 인증 / REST API / React에 데이터 전송 [6p]
- 3대 핵심 아이디어 [7p]
 - 비동기(Asynchronous): 한 작업이 끝나길 기다리지 않음
 - 논블로킹 I/O(Non-Blocking): 대기 중에도 다음 요청 처리
 - 이벤트 드리븐(Event-Driven): 이벤트 발생 시 콜백 실행
- 내부 동작: V8 → Node API → libuv(이벤트 루프 + 워커 스레드) [8p]
- 적합: I/O 중심(REST API, 로그인, 채팅, 실시간) [9p]
- 부적합: CPU 중심(ML 학습, 영상 인코딩, 대규모 연산) → 단일 스레드 이벤트 루프가 막힘 [10p]

✓ 셀프 체크

1. `console.log` 두 줄 사이에 `fs.writeFile` 이 있을 때, 출력 순서는? (왜 그런가?)
2. Node.js가 영상 인코딩 서버로 부적합한 이유를 한 문장으로.
3. Chrome과 Node.js의 공통점과 차이점은?

Node.js 생태계 & 코어 모듈

PRACTICE 1 프로젝트 셋업 & 코어 모듈 [11~22p]

- `npm` = 패키지 매니저, `package.json` = 프로젝트 설정(의존성-scripts), `node_modules` = 설치된 패키지 폴더 [12~16p]
- 코어 모듈(설치 불필요, `require` 만): `fs`(파일), `path`(경로), `http`(서버) [17p]
- `fs.writeFile(파일, 내용, 콜백)` [18p] / `path.join(__dirname, ...)` [20p] / `http.createServer(...).listen(3000)` [22p]



TASK 0 프로젝트 초기화

할 일

1. 작업 폴더에서 `npm init -y` 실행 → `package.json` 생성 [15p]
2. `npm install express`, `npm install mysql` 설치 [13p, 46p]
3. `package.json` 의 `dependencies` 에 `express`, `mysql`이 잡혔는지 확인 [16p]
4. `package.json` 의 `"scripts"` 에 `"start": "node server.js"` 추가 [15~16p]

예상 결과

```
package.json 의 dependencies:
  "express": "^5.x.x",
  "mysql":   "^2.x.x"
node_modules/ 폴더와 package-lock.json 생성됨
```

셀프 체크: `node_modules` 를 깃에 올리면 안 되는 이유? 다시 받으려면? [14p]



TASK 1 코어 모듈로 "파일 로거 + 기본 서버"

할 일 (파일: `task1_core.js`)

1. `fs` 로 `log.txt` 에 `"Server started"` 기록 [18p]
2. 그 경로를 `path.join(__dirname, "log.txt")` 로 만들기 (하드코딩 금지) [20p]
3. `http` 모듈만으로 포트 3000 서버를 띄우고, 모든 요청에 `"Hello from Node.js server"` 응답 [22p]

예상 결과

```
$ node task1_core.js
  → log.txt 파일 생성 (내용: Server started)
브라우저 http://localhost:3000    → Hello from Node.js server
브라우저 http://localhost:3000/abc → (똑같이) Hello from Node.js server ← URL 구분 못 함!
```

힌트 (골격)

```
const fs = require("fs");
const path = require("path");
const http = require("http");

// 1) 로그 파일 경로 만들기
const logPath = path.join(__dirname, /* TODO */);

// 2) 파일 쓰기 (콜백에서 에러 처리)
fs.writeFile(logPath, /* TODO */, function (err) {
  /* TODO */
});

// 3) 서버 생성
const server = http.createServer(function (req, res) {
  /* TODO: res.end(...) */
});
server.listen(/* TODO */);
```

셀프 체크: 출력이 "서버 실행" 로그보다 콘솔의 다른 줄이 먼저 찍힐 수 있다. 왜? [7p]

Express.js

PRACTICE 2 Express 기본 & 라우팅 [23~28p]

- Express = Node 위의 웹 프레임워크. 메서드 검사·URL 매칭·응답 포맷을 자동 처리 [24~25p]
- 최소 구조: `require("express")` → `const app = express()` → `app.get(...)` → `app.listen(3000)` [26p]
- `req` (요청 정보) / `res` (응답: `res.send(...)`) [26p]
- 라우팅 = URL을 핸들러 함수에 매칭. 정의 순서대로 매칭 [27~28p]

🔧 TASK 2 기본 서버를 Express로 다시 만들기

할 일 (파일: `server.js` — 이후 과제는 모두 이 파일에 누적)

1. Task1의 raw http 서버를 **Express로 교체** [25~26p]
2. 라우트 3개 구현 [28p]
 - `GET /` → "Home Page"
 - `GET /products` → "Product List"
 - `GET /users` → "User List"
3. 서버 시작 시 콘솔에 `"Server is running on port 3000"` 출력 [26p]

예상 결과

```
http://localhost:3000/      → Home Page
http://localhost:3000/products → Product List
http://localhost:3000/users  → User List
http://localhost:3000/xxx    → (Express 기본) Cannot GET /xxx ← 이제 URL을 구분함!
```

셀프 체크: raw http 버전(Task1) 대비 코드가 짧아진 이유는? [25p]

PRACTICE 3 HTTP 메서드 / URL 파라미터 / 쿼리스트링 [29~37p]

- `GET` (조회) / `POST` (생성) / `PUT` (수정) / `DELETE` (삭제) [29~32p]
- URL 파라미터 `:id` → `req.params.id` (필수값, 동적 라우트) [33~35p]
- 쿼리스트링 `?key=value` → `req.query.key` (선택값: 검색·필터·정렬) [36~37p]

🔧 TASK 3 User/Product API 라우트 확장

할 일 (`server.js` 에 추가)

1. `GET /users/:id` → `"User ID is " + id` (params) [34p]
2. `GET /products?category=phone` → `"Category is phone"` (query) [36p]
3. `POST /login` → `"Login data received"` (본문 처리는 Task4에서) [30p]
4. `PUT /users/:id` → `"Update user " + id` [31p]
5. `DELETE /users/:id` → `"Delete user " + id` [32p]

예상 결과

```
GET    /users/5           → User ID is 5
GET    /products?category=phone → Category is phone
GET    /products          → Category is undefined ← 쿼리는 선택값
POST   /login             → Login data received
PUT    /users/7           → Update user 7
DELETE /users/7         → Delete user 7
```

POST/PUT/DELETE는 브라우저 주소창으로 테스트 불가. **curl** 또는 **VS Code REST Client / Postman** 사용.

예: `curl -X POST http://localhost:3000/login`

셀프 체크: `:id` (params)와 `?category=` (query)의 차이는? 각각 언제 쓰나? [33p]

PRACTICE 4 미들웨어 & `express.json()` [38~41p]

- 미들웨어 = 요청과 응답 사이에서 실행되는 함수. `next()` 로 다음 단계로 넘김 [38~39p]
- `app.use(fn)` = 모든 라우트에 적용(로그인·인증·파싱) [39p]
- React는 데이터를 **JSON** 텍스트로 보냄 → `app.use(express.json())` 가 파싱해서 `req.body` 객체로 만듦. 없으면 `req.body` 는 `undefined` [40~41p]

🔧 TASK 4 로깅 미들웨어 + 로그인 본문 받기

할 일 (`server.js` 에 추가)

1. 로깅 미들웨어: 모든 요청에 `"[로그] METHOD URL"` 콘솔 출력 후 `next()` [39p]
2. `app.use(express.json())` 추가 [41p]
3. `POST /login` 을 수정: `req.body.email`, `req.body.password` 를 콘솔 출력하고 `"Login data received from " + email` 응답 [41p]

테스트 요청

```
curl -X POST http://localhost:3000/login ^
-H "Content-Type: application/json" ^
```

```
-d '{"email\":\"aziz@cbnu.ac.kr\",\"password\":\"12345\"}'
```

예상 결과

```
(서버 콘솔)  
[로그] POST /login  
aziz@cbnu.ac.kr  
12345  
(응답)  
Login data received from aziz@cbnu.ac.kr
```

셀프 체크: `express.json()` 을 빼면 `req.body` 는 무엇이 되나? 미들웨어에서 `next()` 를 빼면?

[39, 41p]

Node.js & MySQL

PRACTICE 5 DB 개념 & 연결 [42~52p]

- DBMS / 관계형 모델: DB = 테이블(릴레이션)들의 집합, 행=튜플, 열=속성, **기본키**=중복 없는 식별자 [43~45p]
- 드라이버 설치: `npm install mysql` → `var mysql = require("mysql")` [46p]
- 연결: `mysql.createConnection({host, user, password, database})` → `con.connect(cb)` [47, 51p]
- 쿼리 실행: `con.query(sql, function(err, result){ ... })` [48p]
- DB/테이블 생성: `CREATE DATABASE`, `CREATE TABLE` (INT/VARCHAR(n)/DATE/FLOAT) [49~52p]



TASK 5 DB 연결 → 데이터베이스 & 테이블 생성

⚠ 먼저 본인 MySQL 서버의 **user/password**를 준비하세요. 연결 설정은 한 곳(`con`)에 모아 재사용합니다.

할 일 (`server.js` 에 추가)

1. `mysql` require 후 `con = mysql.createConnection({...})` 작성 (처음엔 `database` 없이) [47p]
2. `GET /create-database` → `CREATE DATABASE mydb` 실행 후 `"Database created"` 응답 [49p]
3. 연결 설정에 `database: "mydb"` 추가 [51p]
4. `GET /create-table` → `customers` 테이블 생성 (`id INT PRIMARY KEY`, `name VARCHAR(255)`, `address VARCHAR(255)`) [52p]

예상 결과

```
GET /create-database → Database created   (콘솔: Connected!)
GET /create-table   → Table created
```

힌트 (골격)

```
var mysql = require("mysql");
var con = mysql.createConnection({
  host: "localhost",
  user: /* TODO */,
```



```

password: /* TODO */,
// database: "mydb" ← 3단계에서 주석 해제
});

app.get("/create-table", function (req, res) {
  con.connect(function (err) {
    if (err) throw err;
    var sql = "CREATE TABLE customers (" +
      /* TODO: id, name, address 정의 */
      ")";
    con.query(sql, function (err, result) {
      /* TODO */
    });
  });
});
});

```

셀프 체크: 연결 설정에서 `database` 를 빼고 `CREATE TABLE` 을 하면 나는 에러는? [51p]

PRACTICE 6 CRUD (INSERT / SELECT / UPDATE / DELETE) [53~59p]

- **INSERT:** `INSERT INTO customers (col...) VALUES (val...)` [53p]
- **SELECT:** `SELECT [DISTINCT] 컬럼 FROM 테이블 [WHERE][GROUP BY][HAVING][ORDER BY][LIMIT]` — SELECT/FROM만 필수, 순서 고정 [54p]
 - Node에서 결과 `result` 는 행 객체 배열 → `res.send(result)` 로 JSON 전송 [55p]
 - 세 번째 인자 `fields` = 컬럼 메타데이터(`fields[0].name`) [56p]
- **WHERE:** 조건에 맞는 행만 [57p] ⚠ 사용자 입력을 문자열에 직접 합치지 말 것(SQL 인젝션) → ? 파라미터 사용
- **DELETE:** `DELETE FROM customers WHERE 조건` — WHERE 없으면 전체 삭제 [58p]
- **UPDATE:** `UPDATE customers SET 컬럼=값 WHERE 조건` [59p]

TASK 6 customers 테이블 CRUD 완성

할 일 (`server.js` 에 추가 — 각 라우트 = 강의 예제 1개)

1. `GET /insert-customer` → `(1, 'Company Inc', 'Highway 37')` INSERT, "1 record inserted" [53p]
2. `GET /customers` → `SELECT name, address FROM customers`, 결과 JSON 응답 [55p]
3. `GET /customer-field` → 같은 SELECT 후 `fields[0].name` ("name") 응답 [56p]
4. `GET /customers-by-address` → `WHERE address = 'Park Lane 38'` 결과 [57p]
5. `GET /update-customer` → `SET address='Canyon 123' WHERE address='Valley 345'`, "n record(s) updated" [59p]
6. `GET /delete-customer` → `WHERE address='Mountain 21'`, "Records deleted: n" [58p]

예상 결과

```
GET /insert-customer → 1 record inserted
GET /customers       → [{"name":"Company Inc","address":"Highway 37"}]
GET /customer-field  → name
GET /update-customer → 0 record(s) updated (Valley 345가 없으면 0)
GET /delete-customer → Records deleted: 0
```

셀프 체크

- `result.affectedRows` 는 어떤 값? UPDATE/DELETE에서 왜 유용한가? [58~59p]
- WHERE 없는 DELETE의 위험성은? [58p]



Customers REST API (전 내용 통합)

Part 3(Express·메서드·params·body) + Part 4(MySQL CRUD)를 합쳐 진짜 REST API 하나를 완성합니다. 지금까지는 GET으로만 테스트했지만, 여기서는 메서드와 동작을 올바르게 매칭하고 req.body / req.params로 동적 처리합니다.

요구사항 — customers 테이블 기준

메서드	경로	동작	참고
GET	/api/customers	전체 조회 (SELECT *)	[55p]
GET	/api/customers/:id	한 명 조회 (WHERE id = ?)	[34, 57p]
POST	/api/customers	생성 (req.body INSERT)	[41, 53p]
PUT	/api/customers/:id	수정 (req.body SET, :id WHERE)	[31, 59p]
DELETE	/api/customers/:id	삭제 (:id WHERE)	[32, 58p]

제약 / 도전

- 사용자 입력은 ? 플레이스홀더로 전달 (문자열 직접 결합 금지) [57p]
예: `con.query("SELECT * FROM customers WHERE id = ?", [id], cb)`
- 모든 요청을 로깅 미들웨어로 남기기 [39p]
- `express.json()` 적용 [41p]

예상 동작 (요청 → 응답)

```

POST /api/customers body:{"id":2,"name":"Kim","address":"Seoul"} → 1 record inserted
GET /api/customers → [{id:1,...},{id:2,"name":...}]
GET /api/customers/2 → {id:2,"name":"Kim","address":"Seoul"}
PUT /api/customers/2 body:{"address":"Busan"} → 1 record updated
DELETE /api/customers/2 → Records deleted: 1
  
```

도전 셀프 체크

1. 왜 같은 `/api/customers/:id` 라도 GET·PUT·DELETE가 충돌하지 않는가? (라우팅 기준은?)
2. ? 파라미터가 SQL 인젝션을 막는 원리는?

3. 이 API의 응답을 **React** **useEffect** + **fetch** (Lec12)로 받아 화면에 뿌리려면 어디를 호출하면 되나?

최종 제출 / 완성 체크리스트

- ☐ Task0: `package.json` 에 `express-mysql` 의존성 + start 스크립트
- ☐ Task1: `fs/path/http`로 로그 + 기본 서버
- ☐ Task2: Express 라우팅 3종
- ☐ Task3: `GET(params-query)` / `POST` / `PUT` / `DELETE`
- ☐ Task4: 로깅 미들웨어 + `express.json()` + `/login` 본문 처리
- ☐ Task5: DB-테이블 생성
- ☐ Task6: `INSERT/SELECT/fields/WHERE/UPDATE/DELETE`
- ☐ Capstone: `/api/customers` REST API (? 파라미터, 메서드 매칭)

Questions?

See you next time! 🙌

Part 1 Node 개념 · Part 2 코어 모듈 · Part 3 Express · Part 4 MySQL CRUD
— 전부 직접 구현해 보면 완성!